

Aim

To find the minimum sum of a falling path from the top row to the bottom row of a square matrix using **Dynamic Programming**.

Problem Statement

Given a square matrix, find the minimum sum of a falling path from top to bottom.

At each step, from position (i, j) , movement is allowed to:

- Directly below: $(i+1, j)$
- Diagonally left: $(i+1, j-1)$
- Diagonally right: $(i+1, j+1)$

The objective is to determine the minimum possible path sum.

Input Format

n = matrix size
 $\text{matrix}[][]$ = square matrix

Sample Input

```
3
2 1 3
6 5 4
7 8 9
```

Expected Output

Min Sum = 13

Dynamic Programming Approach

Dynamic Programming is used because the minimum path to a particular cell depends on the minimum paths calculated for the previous row.

Let:

represent the minimum falling path sum ending at position (i, j) .

For the first row:

For every subsequent cell:

Only valid positions are considered when the cell is at the left or right boundary.

DP Table for the Given Example

Input:

```
2 1 3
6 5 4
7 8 9
```

First Row

```
2 1 3
```

Second Row

For 6: $6 + \min(2, 1) = 7$

For 5: $5 + \min(2, 1, 3) = 6$

For 4: $4 + \min(1, 3) = 5$

So:

```
7 6 5
```

Third Row

For 7: $7 + \min(7, 6) = 13$

For 8: $8 + \min(7, 6, 5) = 13$

For 9: $9 + \min(6, 5) = 14$

Final DP table:

```
2 1 3
7 6 5
13 13 14
```

The minimum value in the last row is: 13

Algorithm

1. Read the matrix size n.
2. Read the square matrix.
3. Create a DP table.
4. Copy the first row of the matrix into the DP table.
5. For every remaining row:
 - o Check the upper-left diagonal.
 - o Check the cell directly above.
 - o Check the upper-right diagonal.
 - o Select the minimum.
 - o Add the current matrix value.
6. Find the minimum value in the last DP row.
7. Display the minimum falling path sum.

Source Code

The program is implemented in C using a two-dimensional DP table.

File:

Source_code

Sample Execution

Input

```
3
2 1 3
6 5 4
7 8 9
```

Output

Min Sum = 13

Minimum Path

1 → 4 → 8

Path sum: $1+4+8=13$

Time Complexity

There are $n \times n$ cells in the matrix.

Each cell performs a constant number of comparisons.

Therefore: $O(n^2)$

Space Complexity

The program uses an additional $n \times n$ DP table.

Therefore: $O(n^2)$

Advantages of Dynamic Programming

- Avoids recalculating the same subproblems.
 - Efficient for large matrices.
 - Easy to implement using a bottom-up approach.
 - Reduces the complexity compared with a naive recursive solution.
-

Result

The Minimum Falling Path Sum problem was successfully solved using Dynamic Programming.

For the given matrix:

```
2 1 3
6 5 4
7 8 9
```

the minimum falling path sum is:13